REDUVIA

Personal Finance Tracker

Project Report

Mark D Souza

2907547

Graduate Program, Computer Science

CIS 534 Section 1 Spring ‘26

Prof. Yongjian Fu

Cleveland State University, Ohio

April 17, 2026

Live Application: https://reduvia.vercel.app

Source Code: https://github.com/mxrk2k/reduvia

1. Executive Summary

Reduvia is a full-stack personal finance tracking web application developed as part of the graduate Software Engineering (CIS 534, Spring ’26) coursework. The project started from a straightforward observation: most people either do not track their finances at all, or they use tools that require too much manual effort to be worth it consistently. The goal was to build something that is genuinely easy to use on a daily basis, while also providing the kind of deeper analysis that helps users understand their spending patterns over time.

The application allows users to manually log income and expense transactions, set monthly budgets with intelligent pace-based warnings, and import real bank statements in PDF format for AI-powered financial analysis. It is deployed as a live web application and also has a companion mobile app built with React Native and Expo for iOS and Android.

From a technical standpoint, the project covers the full stack: a Next.js 14 frontend with server-side rendering, a Supabase PostgreSQL backend with row-level security, real-time data visualization using Recharts, AI integration through the Anthropic API for bank statement parsing, and a React Native mobile application sharing the same backend. The result is a production-grade application that is accessible at reduvia.vercel.app.

2. Project Overview

2.1 Background and Motivation

Managing personal finances is something most people know they should do but rarely do consistently. The tools that exist tend to fall into two categories: overly simple (a spreadsheet) or overly complex (enterprise-grade software that requires connecting bank accounts through third-party aggregators). There is not much in between for someone who wants a clean, fast interface for tracking their money without giving a third party access to their banking credentials.

Reduvia was designed to sit in that space. Users can manually enter transactions in under ten seconds, set budgets that proactively warn them when they are spending too fast rather than just showing a static total, and optionally import PDF bank statements to get a complete picture of their finances without any third-party bank connection required.

2.2 Project Objectives

- Build a fully functional, production-deployed personal finance web application
- Implement secure user authentication with protected routes and session management
- Develop a clean, responsive dashboard showing income, expenses, balance, and spending trends
- Create an intelligent budget tracking system that accounts for spending pace within a month
- Integrate the Anthropic API to parse and categorize bank statement PDFs automatically
- Build a companion React Native mobile app sharing the same Supabase backend
- Deploy the application to a live public URL accessible from any device

2.3 Scope

The project covers the complete development lifecycle from initial setup through deployment. This includes database design, backend API logic, frontend UI, mobile development, third-party integrations, and production hosting. The following features were implemented and delivered:

Feature Area	What Was Built
Authentication	Email/password signup, login, logout, email verification, protected routes, middleware-based session management
Transaction Management	Add, view, delete, search, and filter income and expense transactions with categories and descriptions
Dashboard Analytics	Real-time summary cards, income vs. expenses bar chart, monthly trends line chart, spending by category donut chart
Budget Tracking	Per-category monthly budgets with intelligent pace-based warnings that account for where you are in the month
Recurring Transactions	Mark transactions as recurring (weekly, monthly, yearly), due-soon alerts, and automated processing
CSV Export	One-click export of all transactions to a downloadable CSV file
Bank Statement Import	PDF upload with AI-powered parsing and categorization using the Anthropic Claude API
Per-Bank Analysis	Separate analysis pages for each imported bank account with charts and transaction history
Mobile App	React Native Expo app with Dashboard, Transactions, Budgets, and Recurring screens connected to Supabase
Responsive Design	Full mobile and tablet responsiveness across all pages
Dark/Light Mode	Theme toggle with localStorage persistence

3. System Architecture

3.1 High-Level Architecture

Reduvia follows a fullstack architecture centered around Next.js 14 with the App Router. The application is split into three main layers: the frontend user interface, the backend logic layer implemented as Next.js Server Actions, and the data layer managed by Supabase.

The key architectural decision was to avoid building a separate REST API. Instead, Next.js Server Actions handle all data mutations directly on the server, which simplifies the codebase significantly and eliminates the need for separate API route files for common operations like adding or deleting transactions.

3.2 Technology Stack

Layer	Technology
Frontend Framework	Next.js 14 (App Router) with TypeScript
Styling	TailwindCSS with shadcn/ui component library
Backend Logic	Next.js Server Actions (no separate API layer)
Database	Supabase (hosted PostgreSQL)
Authentication	Supabase Auth with email/password and HTTP-only session cookies
Data Visualization	Recharts (line charts, bar charts, pie charts)
3D Animation	Three.js (login page background)
AI Integration	Anthropic API (Claude Sonnet) for bank statement parsing
PDF Parsing	pdf-parse npm package for text extraction
Mobile App	React Native with Expo SDK 54
Mobile Navigation	React Navigation (bottom tabs + stack navigator)
Mobile Charts	Victory Native v36
Deployment	Vercel (web app), Expo Go (mobile development)
Version Control	GitHub (github.com/mxrk2k/reduvia)

3.3 Database Design

The database is hosted on Supabase and uses PostgreSQL. Row Level Security (RLS) is enabled on all tables, ensuring that users can only access their own data regardless of how queries are constructed. The schema consists of seven tables:

- transactions: Stores all manual income and expense entries with support for recurring transaction metadata
- budgets: Stores per-category monthly spending limits linked to the authenticated user
- bank_accounts: Represents an imported bank account identified by bank name and account type
- imported_statements: Records each PDF import with date range, transaction count, opening and closing balances
- bank_transactions: Stores individual transactions extracted from imported PDF statements
- user_preferences: Stores user settings such as whether the onboarding prompt has been dismissed

All tables include a user_id foreign key referencing auth.users, and RLS policies enforce that every SELECT, INSERT, and DELETE operation is scoped to the authenticated user's ID

3.4 Authentication Flow

Authentication is handled entirely by Supabase Auth. When a user signs up, Supabase sends a verification email. After verifying, the user can log in and Supabase sets a session stored in HTTP-only cookies. On every page request, a middleware file checks for a valid session and redirects unauthenticated users to the login page. Authenticated users who navigate to the login page are redirected to the dashboard instead.

4. Feature Implementation

4.1 Transaction Management

The core transaction system allows users to record income and expense entries with a type, amount, category, and description. Each transaction is stored in the transactions table and linked to the authenticated user. The dashboard fetches all transactions server-side on each page load, ensuring the data is always current without requiring client-side state management.

The search and filter system is implemented entirely on the client side for instant responsiveness. When a user types in the search bar or selects a filter, the transaction list re-renders immediately without any network requests. The filter state lives in a client component that receives all transactions as props from the server component.

4.2 Smart Budget Tracking

The budget system goes beyond simply showing how much has been spent against a limit. It incorporates a spending pace calculation based on the current day of the month. For a budget of $300 in a 30-day month, the expected spending by day 10 would be $100. If a user has already spent $200 by day 10, they are spending at twice the expected rate, and the progress bar turns red with a warning message estimating when the budget will be exhausted at the current pace.

This approach gives users actionable information rather than just a static number. A green bar means spending is on track, amber means borderline, orange means ahead of pace, and red means significantly overspending with an estimated exhaustion date.

4.3 Recurring Transactions

Users can mark any transaction as recurring with a frequency of weekly, monthly, or yearly. Recurring transactions appear with a distinct badge in the transaction list. A Due Soon section appears at the top of the dashboard when any recurring transaction is within seven days of its next due date. A process button allows users to generate a new transaction occurrence and advance the next due date automatically.

4.4 Bank Statement Import with AI Parsing

This was the most technically complex feature in the project. Users can upload a PDF bank statement, which is processed through a two-stage pipeline. First, the pdf-parse library extracts the raw text from the PDF. Then, the extracted text is sent to the Anthropic Claude Sonnet API with a structured prompt that asks Claude to identify all transactions, clean up the descriptions, assign categories, determine income versus expense type, and extract metadata such as the bank name, date range, and opening and closing balances.

The advantage of using an AI model for this rather than regex-based parsing is universality. Different banks format their statements in completely different ways, and a regex approach requires custom parsers for each bank. Claude can understand the context of any statement regardless of format and return consistently structured JSON. The parsed transactions are then stored in the bank_transactions table and displayed on a per-bank analysis page with charts and transaction history.

If the API key is not configured or the API call fails, the system falls back to a rule-based keyword categorization so the import still works, just without AI-powered category refinement.

4.5 Mobile Application

The React Native mobile app was built using Expo and shares the same Supabase backend as the web application. The app uses React Navigation with a bottom tab bar for Dashboard, Transactions, Budgets, and Recurring screens. Authentication state is managed via Supabase's session listener, which automatically shows the login screen when no session is present and the main tabs when a session exists.

The Dashboard screen shows the same summary cards as the web app along with income versus expenses bar chart, monthly trends line chart, and spending by category donut chart, all rendered using Victory Native. The Transactions screen supports adding new transactions through a slide-up modal and deleting transactions with a confirmation dialog. The Budgets screen shows the same pace-based progress tracking as the web version.

5. Challenges and Solutions

5.1 PDF Parsing Complexity

The initial approach to parsing bank statement PDFs used regular expressions to detect date and amount patterns in the extracted text. This worked inconsistently because different banks format their statements differently, and even within a single bank's statement, the text extraction by pdf-parse does not always produce clean line breaks. For example, Chase statements concatenate the date directly with the description and the amount directly with the running balance in the raw extracted text.

The solution was to abandon the regex approach entirely and delegate parsing to the Anthropic Claude API. By sending the raw extracted text to Claude with a clear prompt specifying the expected output format, the parsing became reliable across different bank formats without any bank-specific code.

5.2 Supabase Project Pausing

Supabase's free tier pauses projects after a period of inactivity. This caused the production application to become inaccessible after a period of no traffic, returning connection errors that appeared as authentication failures. The fix was to restore the project through the Supabase dashboard and implement a keep-alive mechanism. Additionally, a URL formatting error in the Vercel environment variables was discovered and corrected during this process.

5.3 React Native Version Compatibility

Setting up the Expo mobile app required careful version alignment between the Expo SDK, React Native, and Expo Go on the test device. The initial scaffold used an SDK version that did not match the installed Expo Go app, resulting in incompatibility errors. Resolving this required upgrading to Expo SDK 54 and using the legacy peer dependencies flag during installation to resolve conflicting package version requirements.

5.4 Next.js and pdf-parse Compatibility

The pdf-parse package has a known issue with Next.js where it attempts to load test files during module initialization, causing webpack build failures. This required configuring the package as a server external in next.config.mjs so that webpack excludes it from the bundle and loads it directly at runtime instead.

6. Testing

6.1 Testing Approach

Given the project timeline, testing was conducted manually using a combination of functional testing during development and end-to-end verification on the production deployment. Each feature was tested against the test cases defined in the project's test plan document before being committed to the repository.

6.2 Test Coverage Summary

Test Area	Result
User signup with email verification	Pass
Login with valid credentials	Pass
Login with incorrect credentials shows error	Pass
Unauthenticated access redirects to login	Pass
Logout clears session	Pass
Add income transaction updates summary cards	Pass
Add expense transaction updates summary cards	Pass
Delete transaction recalculates balance	Pass
Transactions persist after logout and re-login	Pass
Row Level Security prevents cross-user data access	Pass
Budget progress bar updates with new expense	Pass
Over-pace warning triggers at correct threshold	Pass
Monthly trends chart renders with multi-month data	Pass
Spending by category chart updates dynamically	Pass
Search filters transaction list in real time	Pass
CSV export downloads correctly formatted file	Pass

PDF import parses Chase statement correctly	Pass
Recurring transaction appears in Due Soon section	Pass
Process Due Transactions advances next due date	Pass
Dark mode persists after page refresh	Pass
Application loads correctly on Vercel production URL	Pass
Mobile app connects to Supabase and displays transactions	Pass

7. Deployment

7.1 Web Application

The web application is deployed on Vercel and connected to the GitHub repository at github.com/mxrk2k/reduvia. Every push to the master branch triggers an automatic build and deployment. The production URL is https://reduvia.vercel.app. Environment variables including the Supabase project URL, Supabase anon key, and Anthropic API key are configured in the Vercel project settings.

7.2 Database

The database is hosted on Supabase's free tier. The schema was created by running seven migration files through the Supabase SQL editor. Row Level Security is enabled on all tables. The Supabase project URL and anon key are used by both the web application and the mobile app to connect to the database.

7.3 Mobile Application

The React Native mobile app is currently distributed through Expo Go for development and testing purposes. The app connects to the same Supabase backend as the web application. Publishing to the Apple App Store and Google Play Store is planned as a post-project milestone and would require enrolling in the Apple Developer Program and Google Play Console.

8. Conclusion

Reduvia came together as a more complete application than originally planned. What started as a basic transaction tracker grew into a full-stack product with AI-powered bank statement analysis, a mobile app, intelligent budget tracking, and recurring transaction management. The project provided hands-on experience with modern web development practices including server-side rendering with Next.js, database security with Supabase RLS, AI API integration, and cross-platform mobile development with React Native.

The most valuable technical lesson from the project was around the limits of rule-based parsing. The initial regex approach to PDF parsing required progressively more complex logic to handle edge cases, and ultimately a single well-crafted prompt to an AI model solved the problem more reliably with far less code. That trade-off between deterministic rule-based systems and AI-based approaches is something that comes up frequently in modern software development, and this project gave a concrete example of when the AI approach is genuinely superior.

The live application is accessible at https://reduvia.vercel.app and the complete source code is available at https://github.com/mxrk2k/reduvia.

Appendix: Project File Structure

The repository is organized as follows:

Path	Purpose
src/app/	All Next.js App Router pages and layouts
src/app/dashboard/	Main dashboard page and components
src/app/budgets/	Budget management page
src/app/import/	Bank statement import page
src/app/bank/[bankAccountId]/	Per-bank analysis page
src/app/(auth)/	Login and signup pages
src/app/actions/	Server Actions for all data mutations
src/components/	Shared UI components
src/lib/	Supabase clients, utilities, PDF parser
src/lib/parsers/	PDF parsing logic
src/types/	TypeScript type definitions
mobile/	React Native Expo mobile application
mobile/screens/	Dashboard, Transactions, Budgets, Recurring screens
mobile/lib/	Supabase client and theme constants for mobile
supabase/migrations/	Database schema SQL files (001-007)
public/	Static assets